

BÁO CÁO THỰC HÀNH LAB 02 BUFFER OVERFLOW TRONG ỨNG DỤNG WEB LOCAL

MSSV: 21127645 - Họ tên: Lê Minh

1. Mục tiêu và môi trường thực hành

Mục tiêu là quan sát lỗi ghi vượt vùng nhớ trong backend native nhận input từ HTTP, dùng ASan/GDB để định vị, xác định các mốc độ dài và kiểm chứng hai bản vá cùng compiler hardening.

Môi trường: Flask tại 127.0.0.1:5002 và GCC/GDB/binary Linux trong Ubuntu, WSL hoặc Docker local. Lab chỉ gây crash có kiểm soát; không có shellcode, ROP hay chiếm quyền.

2. Kịch bản và các bước thực hiện

Gửi tên ngắn để tạo baseline; gửi 64 byte tới profile vulnerable và ghi nhận trạng thái tiến trình; đọc ASan; chạy GDB; kiểm tra nhiều độ dài; thử `secure_length`, `secure_snprintf` và kiểm tra hardening.

ẢNH 01/09

Tên file bắt buộc: 01_normal_input.png

Tiêu đề ảnh: Input bình thường xử lý thành công

Chèn ảnh tại vị trí này.

URL hoặc lệnh: `http://127.0.0.1:5002/vulnerable`

Thao tác: Đã build binary trong WSL/Docker và chạy app local. Nhập Le Minh, profile vulnerable_debug. Bấm Submit/Gửi.

Panel/DevTools: Request Inspector và Native Process Inspector (ghép cùng UI).

Nội dung bắt buộc phải thấy: POST /submit, input ngắn, exit code/stdout thực tế và không có overflow.

Kết quả mong đợi: Chương trình xử lý thành công mà không crash.

Caption: Input bình thường đi qua HTTP và tiến trình native thành công.

Hình 1. Input bình thường đi qua HTTP và tiến trình native thành công.

ẢNH 02/09

Tên file bắt buộc: 02_overflow_http_crash.png

Tiêu đề ảnh: Input dài làm bản vulnerable dừng

Chèn ảnh tại vị trí này.

URL hoặc lệnh: `http://127.0.0.1:5002/vulnerable`

Thao tác: Chọn profile `vulnerable_asan` hoặc `vulnerable_debug`; không dùng `secure`. Nhập 64 ký tự A. Bấm Submit/Gửi.

Panel/DevTools: Request Inspector và Native Process/Final Verdict.

Nội dung bắt buộc phải thấy: `POST length=64` và `exit code/signal/crash` thực tế của tiến trình.

Kết quả mong đợi: Tiến trình `vulnerable` bị ASan dừng hoặc crash; app web vẫn hoạt động.

Caption: Input 64 byte qua HTTP kích hoạt lỗi trong backend native vulnerable.

Hình 2. Input 64 byte qua HTTP kích hoạt lỗi trong backend native vulnerable.

ẢNH 04/09

Tên file bắt buộc: 04_gdb_backtrace.png

Tiêu đề ảnh: GDB tại vị trí lỗi

Chèn ảnh tại vị trí này.

URL hoặc lệnh: `gdb -q -x gdb/inspect_overflow.gdb`

Thao tác: Mở Ubuntu/WSL tại Lab02, đã make all. Nhập Input do script GDB local cung cấp. Chạy lệnh và tiếp tục theo script.

Panel/DevTools: Terminal GDB.

Nội dung bắt buộc phải thấy: Điểm dừng/signal, frame process_name, name[32] hoặc backtrace.

Kết quả mong đợi: GDB định vị lỗi trong stack frame của process_name; không sửa thành ghi.

Caption: GDB xác nhận vị trí lỗi và backtrace của phiên overflow local.

Hình 4. GDB xác nhận vị trí lỗi và backtrace của phiên overflow local.

ẢNH 05/09

Tên file bắt buộc: 05_length_thresholds.png

Tiêu đề ảnh: Mốc capacity, ASan và crash

Chèn ảnh tại vị trí này.

URL hoặc lệnh: `python scripts/test_lengths.py` (hoặc bảng kết quả tích hợp nếu script hiện có).

Thao tác: Đã build các binary trong WSL/Docker. Nhập Các độ dài quanh 31, 32, 40, 64 byte. Chạy lệnh/test và mở bảng kết quả.

Panel/DevTools: Terminal hoặc Length Test table.

Nội dung bắt buộc phải thấy: Capacity 31 byte, mốc ASan đầu tiên và mốc crash đầu tiên từ kết quả thật.

Kết quả mong đợi: Phân biệt ranh giới buffer, phát hiện ASan và crash thực tế.

Caption: Kiểm tra nhiều độ dài xác định capacity, mốc ASan và mốc crash.

Hình 5. Kiểm tra nhiều độ dài xác định capacity, mốc ASan và mốc crash.

3. Nguyên nhân kỹ thuật

Trong `process_name`, `name[32]` là mảng local trên stack frame. `strcpy` sao chép tới byte null nhưng không biết kích thước đích. Input 32 byte đã cần byte null thứ 33; dữ liệu dài hơn có thể ghi đè biến lân cận, canary, saved frame pointer hoặc return address.

`strcpy` và `gets` không nhận capacity đích; `sprintf` không tự giới hạn output. Memory corruption có thể làm hỏng luồng điều khiển và phụ thuộc layout bộ nhớ, nên nghiêm trọng hơn lỗi logic chỉ trả kết quả sai.

ẢNH 03/09

Tên file bắt buộc: 03_asan_log.png

Tiêu đề ảnh: AddressSanitizer định vị ghi vượt buffer

Chèn ảnh tại vị trí này.

URL hoặc lệnh: ASan Inspector hoặc evidence/asan từ lần chạy thật.

Thao tác: Vừa chạy ảnh 02 bằng vulnerable_asan. Nhập Trace 64 ký tự A. Mở ASan Inspector/chi tiết stack trace.

Panel/DevTools: ASan Inspector.

Nội dung bắt buộc phải thấy: stack-buffer-overflow, WRITE, process_name/strcpy, file và dòng liên quan.

Kết quả mong đợi: ASan báo ghi vượt name[32] trong mã vulnerable.

Caption: ASan phát hiện stack-buffer-overflow tại thao tác strcpy.

Hình 3. ASan phát hiện stack-buffer-overflow tại thao tác strcpy.

4. Kết quả và bằng chứng

Bản secure_length đo byte và từ chối trước copy. Bản secure_sprintf giới hạn output theo sizeof(name) và kiểm tra return value để từ chối truncation. Hardening được đọc từ binary thật, không suy diễn từ tên profile.

ẢNH 06/09

Tên file bắt buộc: 06_length_fix.png

Tiêu đề ảnh: Bản vá kiểm tra độ dài

Chèn ảnh tại vị trí này.

URL hoặc lệnh: <http://127.0.0.1:5002/secure/length>

Thao tác: Chọn secure_length. Nhập 64 ký tự A. Bấm Submit/Gửi.

Panel/DevTools: Native/Code Inspector và Final Verdict.

Nội dung bắt buộc phải thấy: strlen/so sánh giới hạn, từ chối trước copy và exit/status thực tế.

Kết quả mong đợi: Không ghi vào buffer khi input vượt 31 byte.

Caption: Bản vá kiểm tra độ dài từ chối input trước thao tác copy.

Hình 6. Bản vá kiểm tra độ dài từ chối input trước thao tác copy.

ẢNH 07/09

Tên file bắt buộc: 07_snprintf_fix.png

Tiêu đề ảnh: Bản vá snprintf xử lý truncation

Chèn ảnh tại vị trí này.

URL hoặc lệnh: <http://127.0.0.1:5002/secure/snprintf>

Thao tác: Chọn secure_snprintf. Nhập 64 ký tự A. Bấm Submit/Gửi.

Panel/DevTools: Native/Code Inspector và Final Verdict.

Nội dung bắt buộc phải thấy: snprintf, sizeof(name), return value và từ chối truncation.

Kết quả mong đợi: Input dài bị từ chối, không được coi là xử lý thành công.

Caption: Bản vá snprintf phát hiện và từ chối kết quả bị cắt ngắn.

Hình 7. Bản vá snprintf phát hiện và từ chối kết quả bị cắt ngắn.

ẢNH 08/09

Tên file bắt buộc: 08_hardening.png

Tiêu đề ảnh: Canary, PIE, RELRO và NX

Chèn ảnh tại vị trí này.

URL hoặc lệnh: <http://127.0.0.1:5002/hardening>

Thao tác: Đã make all, gồm secure_hardened. Nhập Không nhập dữ liệu. Mở/refresh Hardening Inspector.

Panel/DevTools: Hardening Inspector.

Nội dung bắt buộc phải thấy: Canary, PIE, RELRO, NX của profile so sánh; nguồn lệnh kiểm tra binary.

Kết quả mong đợi: Các cơ chế bật/tắt đọc từ binary thật và được mô tả là lớp bổ sung.

Caption: Canary, PIE, RELRO và NX tăng chiều sâu bảo vệ cho binary.

Hình 8. Canary, PIE, RELRO và NX tăng chiều sâu bảo vệ cho binary.

5. Mức độ ảnh hưởng

Input HTTP có thể đi qua Flask tới chương trình C, vì vậy lỗi native có thể bị kích hoạt từ xa đối với service đang phơi endpoint. Hậu quả tối thiểu là tiến trình dừng/DoS; memory corruption còn có thể ảnh hưởng tính toàn vẹn và luồng điều khiển. Lab không khai thác vượt quá crash local.

6. Bản vá và cách phòng chống

Vulnerable tối thiểu: `char name[32]; strcpy(name, user_input)`. Vá 1: dùng `strlen`, yêu cầu độ dài ≤ 31 rồi mới copy và thêm null. Vá 2: `snprintf(name, sizeof name, "%s", user_input)`, sau đó từ chối nếu return value âm hoặc $\geq \text{sizeof name}$.

Giới hạn request ở tầng web; tránh parser C/C++ khi không cần; ưu tiên ngôn ngữ memory-safe. Bật stack protector/canary, PIE kết hợp ASLR, full RELRO, NX và FORTIFY. Đây là lớp giảm thiểu, không thay kiểm tra biên.

7. Trả lời các câu hỏi báo cáo trong BaiTapTopic04.docx

Buffer Overflow là lỗi memory safety do ghi ngoài vùng cấp phát; Injection làm dữ liệu bị interpreter hiểu thành lệnh/cú pháp. Backend native bị kích hoạt qua HTTP vì server chuyển body request thành đối số cho chương trình C.

Firewall không biết capacity của buffer và request hợp lệ về giao thức vẫn có thể chứa input quá dài. Cần sửa code, giới hạn input và hardening ở tiến trình.

Bản vá length hiệu quả vì chặn trước mọi thao tác ghi; bản vá `snprintf` hiệu quả khi vừa giới hạn số byte vừa kiểm tra truncation. Ít nhất ba hardening: canary phát hiện stack bị sửa, ASLR/PIE ngẫu nhiên hóa địa chỉ, NX cấm thực thi vùng dữ liệu; RELRO làm vùng relocation chỉ đọc.

ASLR là ngẫu nhiên hóa layout địa chỉ; DEP/NX ngăn thực thi mã ở vùng dữ liệu; Stack Canary là giá trị kiểm tra trước khi hàm return.

8. Kết quả kiểm thử

Log pytest thật (dòng cuối): 14 passed in 1.45s

Chưa có `length_test.json`; chưa kết luận mốc ASan/crash.

Log GDB thật: chưa có

ẢNH 09/09

Tên file bắt buộc: 09_tests_reports.png

Tiêu đề ảnh: Pytest và report artifacts

Chèn ảnh tại vị trí này.

URL hoặc lệnh: `python -m pytest -q; python scripts/generate_report.py; ls -lh report/`

Thao tác: Chạy trong WSL/Docker nếu test cần binary Linux. Nhập Ba lệnh trên. Chạy lần lượt.

Panel/DevTools: Terminal.

Nội dung bắt buộc phải thấy: Tổng kết pytest thực tế, tên DOCX/PDF và kích thước khác 0.

Kết quả mong đợi: Chỉ ghi đạt khi pytest exit 0; report artifacts tồn tại.

Caption: Kết quả pytest thực tế và report artifacts của Lab02.

Hình 9. Kết quả pytest thực tế và report artifacts của Lab02.

9. Kết luận

Lỗi bắt nguồn từ copy không có giới hạn vào `name[32]`. Kiểm tra biên và xử lý truncation là bản vá chính; ASan/GDB giúp phát hiện và phân tích, còn hardening chỉ giảm khả năng/tác động khi lỗi vẫn tồn tại.